

福州大学

编译系统设计实践

学 生 姓 名 : 张三

学 生 学 号 : 102301789

专 业 班 级 : 计算机科学与技术

指 导 教 师 : 何振峰

2026 年 6 月 18 日

目录

1	词法分析器	3
1.1	整体架构与字符扫描	3
1.2	多字符运算符与关键字识别	3
1.3	字符串与字符常量	4
1.4	Token 输出与错误处理	4
1.5	设计反思（可选）	4
2	语法分析器	4
2.1	递归下降 Parser 的设计	4
2.2	特殊语法结构的处理	5
2.3	错误处理策略	5
2.4	与 LL(1) 文法的关系（可选）	5
3	语义分析	5
3.1	符号表设计	5
3.2	类型推断规则	6
3.3	类型检查	6
3.4	作用域与变量查找（可选）	6
4	Mandrill 编译器	6
4.1	中间代码与代码生成策略	7
4.2	控制流翻译	7
4.3	函数调用的翻译	7
4.4	数组与字符串的处理	7
4.5	汇编模拟器执行	8
5	心得体会	8
6	总结	9
A	AI 使用情况说明	10
A.1	使用声明	10
A.2	详细使用记录	10
A.3	诚信承诺	11

摘 要

请在这里撰写摘要，简要介绍你在编译原理课程中实现 Mandrill v2.0 语言编译器的完整过程。建议包含以下内容：

- 你使用的编程语言与构建工具；
- 词法分析器和语法分析器采用的技术路线（按课程要求应为手写递归下降实现）；
- 语义分析器与编译器后端是否借助了 ANTLR；
- 编译器输出的目标代码形式（文本汇编指令），以及由课程组提供的模拟器执行；
- 测试通过情况（合法用例与语义错误用例）。

摘要应控制在 200–300 字左右，语言简洁凝练。

1 词法分析器

【本章写作提示】 请按课程要求，描述你手写实现的词法分析器。以下给出建议的写作结构与参考内容，请根据你的实际实现进行补充或调整。

1.1 整体架构与字符扫描

请描述你的词法分析器采用的核心 IO 组件（如 `PushbackReader`、`BufferedReader` 等），以及如何逐字符读取源代码。建议说明以下技术细节：

- 如何维护行号与列号（遇到换行符时的处理）；
- 扫描每个 Token 前如何记录起始位置；
- 如何处理 Windows 换行符 `\r\n` 或 `\r` 的兼容性。

1.2 多字符运算符与关键字识别

请描述单字符运算符与多字符运算符的识别逻辑。对于多字符运算符（如 `==`、`!=`、`<=`、`>=`），请说明你如何实现向前看（look-ahead）机制。

关键字的识别建议通过查找表（如 `HashMap<String, TokenType>`）实现，请说明你的设计方案及其时间复杂度优势。

1.3 字符串与字符常量

请描述字符串常量与字符常量的解析逻辑，包括但不限于：

- 转义字符（`\\`、`\n`、`\'` 等）的处理方式；
- 未闭合字符串或字符常量的错误检测与报告。

1.4 Token 输出与错误处理

请说明你的 Token 输出格式，以及遇到非法输入（如非法字符、未闭合字符串）时的错误处理策略（是否输出错误信息、是否安全退出等）。

词法分析器需要识别的 Token 类别可参考 [表 1](#) 整理。你可以直接使用或修改下表。

表 1: Mandrill 词法 Token 类别（示例）

类别	具体符号
关键字	<code>if, else, while, read, put, write, get, func, global, local, return, break, continue</code>
运算符	<code>+, -, *, /, %, <, <=, >, >=, ==, !=, =</code>
分隔符	<code>(,), [,], {, }, ,, ;</code>
字面量	整数、字符常量、字符串常量
标识符	<code>[a-zA-Z_][a-zA-Z0-9_]*</code>

1.5 设计反思（可选）

请补充你在手写词法分析器过程中的感悟。例如：哪些细节花费了比你预期更多的时间？手写实现与自动化工具相比有何优劣？

2 语法分析器

【本章写作提示】 请描述你手写实现的语法分析器。以下给出建议的写作结构与参考内容。

2.1 递归下降 Parser 的设计

请描述你为每个非终结符编写的递归下降方法，以及如何处理 Mandrill EBNF 文法。建议重点说明：

- 表达式优先级分层的设计（如何自然地消除左递归并处理优先级）；
- 以 `a + b * c` 为例，解释分层结构如何保证 `b * c` 被优先解析。

以下是一段产生式分层结构的 LaTeX 示例，你可以直接引用或替换为你自己的文法：

```
Expression      ::= LogicalExp
LogicalExp      ::= RelationalExp { ("==" | "!=") RelationalExp }
RelationalExp   ::= AdditiveExp { ("<" | "<=" | ">" | ">=") AdditiveExp }
AdditiveExp     ::= MultiplicativeExp { ("+" | "-") MultiplicativeExp }
MultiplicativeExp ::= Primary { ("*" | "/" | "%") Primary }
```

2.2 特殊语法结构的处理

请说明你在实现中遇到的特殊语法结构及其处理方式，例如：

- 数组初始化语句 `a[] = expr`：如何通过向前看多个 Token 区分数组初始化与数组元素访问？
- `if-else` 语句的 `dangling-else` 歧义：你采用了哪种匹配策略？
- 空语句；的处理方式。

2.3 错误处理策略

请描述你的语法分析器如何处理语法错误。例如：是否先检查词法错误？遇到非预期 Token 时如何报告？是否实现了错误恢复机制（如 `panic mode`）？

2.4 与 LL(1) 文法的关系（可选）

请分析 Mandrill 文法是否满足 LL(1) 条件。对于每个非终结符的多个产生式，它们的 FIRST 集是否互不相交？你如何通过首 Token 唯一确定产生式？

3 语义分析

【本章写作提示】 请描述你基于 ANTLR 实现的语义分析器。以下给出建议的写作结构与参考内容。

3.1 符号表设计

请描述你设计的符号表数据结构。建议说明以下三层结构及其字段：

- **全局变量**：名称、类型、全局数据区编号等；
- **函数定义**：名称、参数列表、返回值类型、入口标签、局部变量表等；

- **局部变量**：名称、类型、栈帧偏移等。

请说明你是否采用两遍扫描策略（先收集签名再检查函数体），以及为什么需要这样做（如 Mandrill 支持函数前向调用）。

3.2 类型推断规则

Mandrill 具有隐式类型推断特性。请整理并描述你实现的类型推断规则，例如：

- `a = expr` 时如何根据 `expr` 的类型推断 `a` 的类型；
- `a[] = expr` 时 `a` 的类型推断；
- `local a;` 与 `local a[];` 的区别；
- 未声明直接使用的变量的默认推断规则。

3.3 类型检查

请描述你的类型检查器实现了哪些检查规则。建议包括但不限于：

- 赋值左右类型兼容性；
- 数组索引类型与被索引对象类型；
- `break/continue` 是否仅出现在循环体内；
- `return` 返回值与函数声明的一致性；
- 函数调用实参与形参的个数及类型匹配。

请说明错误报告机制：如何携带行列号信息？输出格式是什么？

3.4 作用域与变量查找（可选）

请描述 Mandrill 的作用域规则以及你的变量查找策略。例如：函数内部如何访问全局变量？局部变量、函数参数、全局变量的查找顺序是什么？

4 Mandrill 编译器

【本章写作提示】 请描述你基于 ANTLR 实现的编译器。以下给出建议的写作结构与参考内容。

4.1 中间代码与代码生成策略

请描述你的编译器采用的代码生成策略。例如：

- 是直接由 AST 生成汇编，还是经由中间表示（如三地址码 TAC）再生成汇编？
- 若使用了 TAC，请列出你实现的 TAC 指令种类；
- 临时变量的分配策略是什么？

4.2 控制流翻译

请描述控制流语句的翻译方案。建议画出或说明以下结构生成的汇编代码模板：

- **while** 循环：条件判断、循环体、跳转回起始标签的机制；
- **if-else**：条件判断、then 分支、else 分支、合并标签；
- **break** 与 **continue**：如何通过标签栈实现目标地址回填？

请特别注意 **eval** 条件跳转的栈操作顺序：先压入 **ELSE** 地址，再压入 **THEN** 地址，最后压入条件值（**cond** 在栈顶）。

4.3 函数调用的翻译

请描述函数调用的完整汇编生成过程。建议说明：

- 调用者如何压入参数；
- **jal** 指令的作用（保存 SP 与 PC）；
- 被调用者如何将参数从栈中取出并存入局部变量；
- 返回值如何传递；
- **ret** 指令如何恢复调用者上下文；
- 无返回语句的函数是否需要特殊处理（如默认返回 0）。

4.4 数组与字符串的处理

请描述编译器如何处理数组与字符串：

- 数组元素访问的地址计算公式（如 $\text{Address} = \text{Base}(a) + i \times 4$ ）；
- 字符串常量的内存分配与初始化过程（**malloc**、逐字符写入、UTF-32 编码、0 结尾）；

- 数组赋值与字符串赋值在代码生成上的区别。

编译器生成的核心指令集可参考 表 2 整理。

表 2: Mandrill 虚拟机汇编指令集（示例）

类别	指令	说明
数据存取	<code>dstore x</code>	将立即数 <code>x</code> 压入操作数栈
	<code>dload x</code>	从全局变量区读取变量 <code>x</code> 入栈
	<code>dlload x</code>	从局部变量区（相对 SP 偏移 <code>x</code> ）读取入栈
	<code>dlwrite x</code>	将栈顶元素写入局部变量 <code>x</code> 并弹出
	<code>daload x</code>	从栈顶地址读取数组元素入栈
	<code>dawrite x</code>	将值写入栈顶地址指向的内存
运算	<code>eval x</code>	根据操作数 <code>x</code> 执行二元运算或条件跳转
跳转	<code>jump x</code>	将 PC 设置为 <code>x</code>
	<code>jal x</code>	链接跳转，保存上下文并调用函数
	<code>ret x</code>	恢复调用者上下文并返回
内存	<code>malloc x</code>	从栈顶获取大小，申请堆内存并将地址入栈
I/O	<code>geti/getc/gets</code>	读取整数/字符/字符串并入栈
	<code>puti/putc/puts</code>	弹出栈顶并输出到 <code>stdout</code>

4.5 汇编模拟器执行

请描述实验四的完整执行流程。注意：模拟器由课程组提供，你不需要实现它，但需要理解其工作原理以正确生成汇编。建议说明：

- 编译器从标准输入读取源程序，输出文本汇编到标准输出；
- 评测机如何将汇编重定向到临时文件并由模拟器加载执行；
- 模拟器的文本汇编解析方式（逐行解析、注释处理、操作数格式）；
- 模拟器的运行时内存模型（操作数栈、全局变量区、局部变量区、调用栈帧、堆内存）；
- 理解模拟器对你正确生成汇编有何帮助。

5 心得体会

【本章写作提示】 请结合你的实际实现过程，撰写真实的感悟与反思。以下方向供参考：

- 手写词法分析器和语法分析器时，哪些细节花费了比你预期更多的时间？（如行列号维护、向前看机制、dangling-else 处理等）
 - 从手写前端转向 ANTLR 实现语义分析与代码生成，你有何体会？工具带来了哪些便利，又有哪些原理性的东西必须手动处理？
 - 编译器后端中，控制流跳转、函数调用约定、栈操作管理是否是难点？你是如何理清思路的？（如手绘栈状态图、单步跟踪等）
 - 中间表示（如 TAC）是否帮助你解耦了前端与后端？
 - 项目采用 Gradle 多模块管理，你在模块划分、依赖管理、独立测试方面有何体会？
- 请避免空洞的套话，尽量写出具体、真实的技术反思。

6 总结

【本章写作提示】 请用 2-3 段话总结你的项目成果。建议包含：

- 你完成了哪些模块的实现；
- 采用了哪些关键技术；
- 测试通过情况；
- 是否达到了课程实践的教学目标。

致谢

【本章写作提示】 请写上对你实现该项目帮助较大的同学、学长学姐、大模型或课程组教师。请保持真诚简洁。

A AI 使用说明

【必填】根据课程学术诚信要求，请如实填写你在完成本实验过程中使用 AI 辅助的情况。若未使用任何 AI 工具，请明确写明“本次实验未使用任何 AI 工具”。

A.1 使用声明

项目	填写内容
是否使用 AI 工具	是 / 否
使用的 AI 模型	(如: GPT-4、Claude 3.5、Kimi、Copilot、通义千问等)
使用场景	(如: 代码调试、算法设计、报告撰写、LaTeX 排版、概念解释等)
使用频率	(如: 偶尔、经常、全程辅助等)

A.2 详细使用记录

请按时间顺序或按模块列出你使用 AI 的具体情况。建议每条记录包含以下要素：

1. **使用场景**：你在做什么时使用了 AI？
2. **输入提示词 (Prompt)**：你向 AI 提出的具体问题或指令（请尽量如实还原）。
3. **AI 输出摘要**：AI 给出了什么样的回答或建议？
4. **你的采纳与修改**：你是否直接采用了 AI 的输出？如果进行了修改，请说明原因。

填写示例（请删除此文本框后如实填写）：

场景：编写递归下降语法分析器时遇到 dangling-else 问题。

Prompt：“如何在递归下降 parser 中处理 if-else 的 dangling-else 歧义？请给出 Java 代码示例。”

AI 输出：建议采用贪心策略，让 else 匹配最近的未匹配 if。

采纳情况：参考了该思路，但根据自己的文法结构重新实现了代码。

请在此处开始填写你的 AI 使用记录：

记录 1

场景	
Prompt	
AI 输出摘要	
采纳与修改	

记录 2

场景	
Prompt	
AI 输出摘要	
采纳与修改	

A.3 诚信承诺

本人承诺：以上关于 AI 使用情况的说明真实、完整。本人理解并遵守课程关于学术诚信的相关规定，未将 AI 生成内容未经审阅直接作为自己的成果提交。

签名：_____ 日期：_____